Análisis técnico y decisiones

NEXUS — Propuesta de evolución desde H3 | Octubre de 2026

1. Punto de partida

NEXUS H3 es un juego de consola C# con clases orientadas a objetos. La meta es construir una interfaz gráfica 2D, conservar reglas y guardar partidas. El análisis de código se basa en el archivo aportado previamente, no en una ejecución del repositorio actual.

2. Interfaz: consola frente a Windows Forms

Consola: simple y ya implementada, pero no ofrece exploración visual. Windows Forms: cumple la consigna, integra controles y dibujo 2D, pero no es un motor de videojuegos; animaciones, temporización y colisiones requieren desarrollo manual. Decisión: Windows Forms con un mapa pequeño, gráficos sencillos y actualización controlada.

3. Persistencia: JSON frente a SQL Server

JSON: útil para prototipos, simple y portátil; requiere resolver integridad y sincronización manualmente. SQL Server: relaciones, transacciones y consultas robustas; exige instalación y configuración. Decisión: SQL Server, preferiblemente edición de desarrollo local compatible con el entorno, por ser obligatorio.

4. Acceso a datos: ADO.NET

Se recomienda Microsoft.Data.SqlClient con parámetros SQL y repositorios separados. Ventaja: control explícito y poca abstracción para un proyecto educativo; desventaja: mayor cantidad de código de mapeo y SQL. Una alternativa futura es Entity Framework Core, evaluando compatibilidad y complejidad.

5. Arquitectura: monolito acoplado frente a capas

Mantener Console dentro de SistemaNexus haría difícil crear una GUI y probar reglas. Separar presentación, dominio y datos permite reutilizar Cadete, Mision, Inventario y encuentros, sin reescribir las reglas cada vez que cambie la pantalla. Se recomienda refactorización incremental.

6. Riesgos y mitigación

Acoplamiento a consola: extraer reglas y devolver resultados sin imprimir.
Colisiones y rendimiento: mapa pequeño, pocos elementos y repintado limitado.
Cambios en reglas de combate: pruebas unitarias de daño y energía.
Pérdida de progreso: transacciones y pruebas de guardar/cargar.
Credenciales expuestas: configuración local excluida de Git.
Alcance excesivo: entregar primero una misión completa y cuatro zonas básicas.

7. Pruebas y evidencia planificadas

Prueba 1: cadete inicia con 100 vida y 60 energía.
Prueba 2: GastarEnergia y RecuperarEnergia respetan límites.
Prueba 3: encuentro con Robot/Enemigo produce los daños previstos.
Prueba 4: inventario rechaza índice inválido.

Prueba 5: los cuatro objetivos son necesarios para completar misión.
Prueba 6: guardar y cargar restaura inventario, zona, vida y objetivos.
Estas pruebas son propuestas, no resultados ejecutados.

8. Verificación de IA y fuentes

La IA apoyó la redacción y comparación de alternativas. Las afirmaciones sobre el H3 se contrastaron con el archivo C# previamente compartido; no se afirma haber comprobado el estado actual de GitHub. Las decisiones técnicas deben verificarse durante implementación con documentación oficial y pruebas de compatibilidad.

Microsoft Learn — Windows Forms: https://learn.microsoft.com/dotnet/desktop/winforms/
Microsoft Learn — C#: https://learn.microsoft.com/dotnet/csharp/
Microsoft Learn — SQL Server: https://learn.microsoft.com/sql/sql-server/
Microsoft Learn — ADO.NET: https://learn.microsoft.com/dotnet/framework/data/adonet/
Microsoft Learn — Arquitectura .NET: https://learn.microsoft.com/dotnet/architecture/

9. Recomendación final

Priorizar una versión vertical mínima: menú, un cadete, cuatro zonas, misión original, inventario, combate y persistencia. Mantener los archivos de H3 y commits descriptivos. Los documentos definen la futura implementación, no demuestran que la GUI o la base de datos ya existan.